



KILID MESSENGER

Software Architecture Document

Formal Technical Reference · Version 1.0.0

Document Title	KILID Messenger — Software Architecture Document
Version	1.0.0
Status	Released
Classification	Technical Reference
Scope	Desktop client architecture, security, data, transport
Platform	Windows · Linux · macOS (Python / PyQt6)
Transport	MQTT over TLS (public or private brokers)
Crypto	ECDH SECP384R1 · AES-256-GCM · PBKDF2-HMAC-SHA512
Repository	github.com/armanghobadi/kilid
Website	https://kilid.app

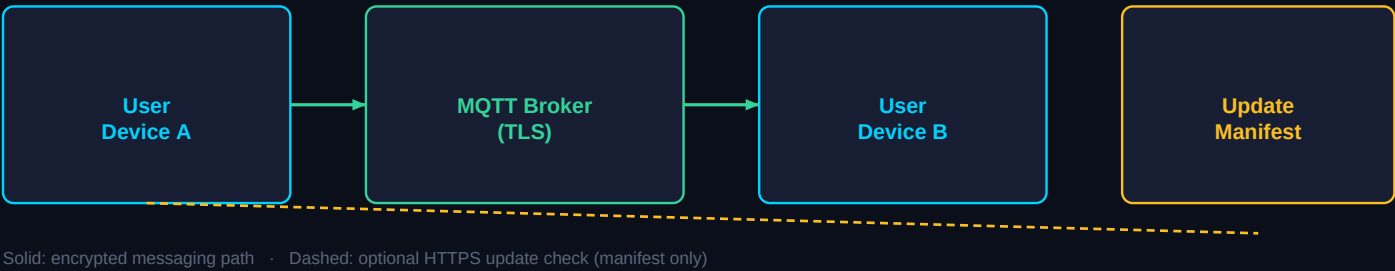
This document describes the internal structure, interfaces, security model, data stores, and runtime behaviour of the KILID desktop messenger client.

1. Table of Contents

- 1. Document Control & Contents
- 2. System Context & Stakeholders
- 3. Logical Architecture (Layered View)
- 4. Component Catalogue
- 5. Security Architecture & Cryptography
- 6. Identity, Authentication & Recovery
- 7. Messaging Protocol & MQTT Topics
- 8. Session Keys, Ratchet & Control Frames
- 9. Database Schema & Local Storage
- 10.Group Encryption Model
- 11.Runtime, Threading & Reliability
- 12.UI Structure & Navigation Map
- 13.Games Subsystem
- 14.Threat Model, Limits & Compliance Notes

2. System Context

KILID is a pure client application. It does not operate a proprietary message store. End-user devices communicate through one or more MQTT brokers. Message plaintext, private keys, and recovery material never leave the device unencrypted.



3. Logical Architecture — Layered View

The client is organised into five logical layers. Dependencies point downward only. Upper layers must not bypass the Security layer when handling message content or keys.

L5 Presentation

PyQt6 widgets, dialogs, themes, system tray, games boards, login shell

KilidApp (QMainWindow) · LoginDialog · Chat / Group / Settings pages · BackgammonBoard · ChessBoard

L4 Application Services

Orchestration of messaging, presence, invites, file transfer, notifications

MessageRouter · PresenceManager · GroupService · NotificationService · FileTransferService

L3 Security

All cryptographic operations, key storage, verification, recovery blobs

CryptoEngine · SessionKeyStore · SafetyNumber · Backup / Transfer crypto

L2 Persistence

Thread-safe SQLite access, migrations, connection pool, maintenance

DatabaseManager · Schema v6 · WAL + pool · Cleanup jobs

L1 Transport

MQTT client lifecycle, TLS, topic pub/sub, reconnect, discovery envelope

MqttManager · Topic namespace kilid/ · Retained transfer topics

4. Component Catalogue

DatabaseManager

L2

Owns SQLite path `~/.kilid/data/<user>.db`. Connection pool (max 10), RLock, migrations to `DATABASE_VERSION=6`, background cleanup of nonces/sessions/invites.

MqttManager

L1

paho-mqtt client. Connect/reconnect with exponential backoff, TLS via certifi, publish/subscribe under prefix `kilid/`, discovery wrap/unwrap with HMAC.

Crypto / Session helpers

L3

ECDH SECP384R1, AES-GCM encrypt/decrypt, PBKDF2 (600k iters), HKDF, recovery blob encrypt, group PIN blob, control HMAC version 1.

LoginDialog

L5

Frameless modal: login, register (Terms), restore (Safety Number / `.kilidbak` / online). Background threads + QueuedConnection slots for UI safety.

KilidApp (Main Window)

L5/L4

Central orchestrator after login: loads contacts, wires MQTT callbacks, hosts stacked pages (chat, groups, settings, games), tray icon.

Message pipeline

L4/L3

Outbound: `serialize` → `encrypt` → MQTT publish. Inbound: `decrypt` → `validate timestamp/nonce/rate` → `persist` → UI signal. Supports text, voice, file chunks, control.

GroupService

L4

Create/join, invite codes, PIN-encrypted join secrets, member/admin lists, shared `group_key` distribution and storage in `group_keys` table.

GamesController

L5/L4

BackgammonEngine / ChessEngine, board widgets, online invite/accept/state over `msg_type` game, seeking broadcast to contacts.

LoggerManager

Cross-cutting

Singleton rotating file + console handlers under `~/.kilid/logs/kilid.log` (security, system, audit, network, voice, block channels).

5. Security Architecture & Cryptography

5.1 Cryptographic Primitives

■ Key generation	EC SECP384R1 (identity key pair)
■ Key agreement	ECDH → shared secret
■ KDF (password)	PBKDF2-HMAC-SHA512, 600 000 iterations, 32–64 byte output
■ KDF (session)	HKDF-SHA256 / application-specific derive from ECDH
■ Confidentiality + integrity	AES-256-GCM (12-byte IV, 16-byte tag)
■ MAC (discovery / control)	HMAC-SHA256 with application salt
■ Addressing	HMAC-SHA256 truncated to 32 hex chars (peer MQTT address)
■ Randomness	secrets / os.urandom for salts, nonces, PINs, tokens

5.2 Key Hierarchy

User password → PBKDF2 → (a) password_hash for login verification, (b) derived key to encrypt private key PEM at rest. Identity private key + peer public key → ECDH → session/shared key material → AES-GCM for chat packets. Group: random group_key shared among members. Recovery Safety Number → PBKDF2 → AES-GCM recovery_blob of plaintext private key.



5.3 At-Rest Protection

Private key stored as PKCS8 PEM encrypted with BestAvailableEncryption using password-derived key. Recovery blob: AES-GCM with key from 32-digit Safety Number. SQLite file is not full-disk encrypted by the app; OS-level disk encryption is recommended.

5.4 Hardening Parameters (constants)

MESSAGE_TIMESTAMP_WINDOW	±180 s	KEY_EXCHANGE_TIMESTAMP_WINDOW	±120 s
MAX_PACKET_SIZE	512 KB	RATE_LIMIT_PER_SENDER	60 / 10 s
RATE_LIMIT_GLOBAL	300 / 10 s	KEY_ROTATE_EVERY_N_MESSAGES	50
KEY_ROTATE_MAX_AGE_SEC	3600	RATCHET_SKIP_LIMIT	64
PBKDF2_ITERATIONS	600 000	MAX_FILE_SIZE	10 MB
CHUNK_SIZE	128 KB	MAX_VOICE_DURATION	60 s

6. Identity, Authentication & Recovery

6.1 Identifiers

On registration the client derives a stable machine-related `mac_id` and a `user_id` from `username`. `combined_id` is formed as `mac_id:user_id`. Public MQTT addressing uses `kilid_peer_addr(combined_id)`: HMAC-SHA256 with `KILID_ADDR_SALT`, truncated to 32 hexadecimal characters. Wire addresses are never double-hashed if already 32-hex.

6.2 Registration Sequence

1. Validate username uniqueness (local DB file existence / users table).
2. Generate EC SECP384R1 key pair; export public key (DER SubjectPublicKeyInfo).
3. PBKDF2(password) → password_hash + salt; separate salt encrypts private key PEM.
4. Generate 32-digit Recovery Safety Number; store only hash + AES-GCM recovery_blob.
5. Persist users row (broker_config JSON, device_name, security_level=high).
6. Display Safety Number once; user must store offline.

6.3 Login Sequence

1. Open DatabaseManager(username); SELECT password_hash, salts, keys, broker_config.
2. Recompute PBKDF2(password, salt); secrets.compare_digest against stored hash.
3. On success: decrypt private key, normalize broker_config (TLS defaults for known hosts).
4. Failed attempts counter per username; lock after LOGIN_MAX_PASSWORD_ATTEMPTS (5).
5. Emit login_successful → parent constructs KilidApp with db handle and password material.

6.4 Recovery Paths

Safety Number

- 32 digits, offline
- Verifies recovery_safety_hash
- Decrypts recovery_blob
- Re-encrypts privkey with new password
- Updates password_hash

Backup File

- .kilidbak container
- Password-derived AES-GCM
- Restores account payload
- Recreates local DB user
- Then normal login

Online Transfer

- 128-char transfer id
- 16-digit safety PIN
- Retained MQTT topic
- Envelope KILID_TRANSFER_V1
- Expiry check + decrypt

7. Messaging Protocol & MQTT Topics

7.1 Topic Namespace

All application traffic is prefixed with MQTT_TOPIC_PREFIX = "kilid" to reduce collision on public brokers. Peer addresses are 32-character hex digests. Account transfer uses retained messages under kilid/account_transfer/<sha256(transfer_id)>.

<code>kilid/<peer_addr>/...</code>	Direct encrypted chat / control to a peer
<code>kilid/.../discovery</code>	Wrapped presence / seek (HMAC envelope)
<code>kilid/account_transfer/<hash></code>	Retained online restore packages

7.2 Discovery Envelope

kilid_discovery_wrap: JSON payload → base64 → outer {v:1, data, hmac} where hmac is HMAC-SHA256(KILID_ADDR_SALT, raw). Unwrap verifies compare_digest before parse. Legacy plain JSON still accepted for migration.

7.3 Message Type Catalogue (MessageType enum)

<code>text</code>	Standard chat body
<code>file / file_info / file_chunk</code>	Metadata + chunked binary transfer
<code>voice</code>	Compressed audio segments
<code>image / video / location / contact</code>	Rich media / share
<code>system</code>	Local system notices
<code>typing</code>	Ephemeral typing indicator
<code>read_receipt / delivery_receipt</code>	Control receipts
<code>reaction / delete / edit</code>	Message lifecycle
<code>poll / buzz</code>	Interactive / attention
<code>group_invite / group_join_request</code>	Group membership
<code>profile_request / profile_response</code>	Avatar & bio exchange
<code>game</code>	Chess / Backgammon state & invites

7.4 Broker Configuration

normalize_broker_config() applies defaults: host broker.hivemq.com, port 8883, tls=True, ca_certs=certifi.where(). Known hosts (HiveMQ, Mosquitto, EMQX) auto-upgrade legacy plain 1883 configs to TLS. Custom brokers allow user/pass and explicit CA path.

8. Session Keys, Ratchet & Control Frames

8.1 Peer Session Establishment (TOFU)

1. Contact added with node_id (combined_id or wire address).
2. Key exchange messages carry public keys + timestamps within KEY_EXCHANGE_TIMESTAMP_WINDOW.
3. ECDH produces shared secret; application derives AES-GCM key material.
4. Public key of peer stored (contacts / users placeholder); first-use trust (TOFU).
5. Users should compare Safety Numbers out-of-band for high-assurance links.
6. Rate limit: RATE_LIMIT_KEYEX_PER_PEER = 5 attempts / minute.

8.2 Ratchet / Rotation Policy

After KEY_ROTATE_EVERY_N_MESSAGES (50) or KEY_ROTATE_MAX_AGE_SEC (3600), session material is re-derived. RATCHET_SKIP_LIMIT (64) bounds retained out-of-order message keys. This limits the window of compromise for past traffic when forward secrecy is required.

8.3 Inbound Validation Pipeline

Size	Reject if encrypted packet > MAX_PACKET_SIZE (512 KB)
Timestamp	Drop if outside MESSAGE_TIMESTAMP_WINDOW (±180 s)
Nonce	Minimum decoded length MIN_NONCE_BYTES (24); track used nonces
Rate	Per-sender and global token buckets
Decrypt	AES-GCM; InvalidTag → discard
AuthZ	Block list check; group membership for group traffic
Persist	Insert messages row; update conversation metadata
Signal UI	Qt signals to chat view / notifications

8.4 Control Frames

Delivery and read receipts, typing indicators use compact control payloads (MAX_CONTROL_PAYLOAD 64 KB) with CONTROL_HMAC_VERSION for authenticity without full chat encryption overhead where appropriate. Buzz and voice sends enforce per-peer cooldowns (10 s).

9. Database Schema & Local Storage

Path: ~/.kilid/data/<username>.db · PRAGMA journal_mode=WAL, synchronous=NORMAL, foreign_keys=OFF, busy_timeout=5000, cache_size≈20MB. Schema version via PRAGMA user_version (current DATABASE_VERSION = 6).

users

user_id PK, username, password_hash/salt, combined_id, private_key/salt, public_key, broker_config, avatar, bio, status, recovery_safety_hash/salt, recovery_blob/salt

contacts

alias, node_id, user_id, public_key, is_blocked, is_favorite, notes

groups

group_id, name, creator_id, group_key, password_hash, invite_code, is_private, metadata

group_members

group_id, user_id, is_admin, nickname, joined_at

group_invites

invite_code, sender_id, expires_at, max_uses, used_count, password

group_keys

group_id, user_id, group_key (per-member copy)

messages

message_id PK, conversation_id/type, content, sender/receiver, msg_type, flags (sent/read/delivered/deleted), file_*, reply_to, reactions, metadata

shared_keys

user_id, peer_id, shared_key, session_key, expires_at

block_list

user_id, blocked_user_id, block_type, reason, is_active

voice_messages

message_id, file_path, duration, is_played, file_size

nonces

nonce PK, timestamp, expires_at (replay defence)

sessions / devices / notifications / stats / user_settings / message_reactions

Supporting operational and UI state

9.1 Migration Path

v1 baseline → v2 profile columns → v3 reactions/metadata → v4 block/devices/invites/group_keys/voice/notifications/stats/sessions → v5 ensure group columns → v6 recovery_safety_* and recovery_blob_* on users.

10. Group Encryption Model

10.1 Shared Group Key

Each group holds a random `group_key` (bytes). Members receive a copy stored in `group_keys`. Group messages are encrypted with this key (AES-GCM). Creators and admins manage membership; `password_hash` uses PBKDF2-SHA512 (120k iterations) or legacy SHA-256 for migration.

10.2 Invite Mechanisms

Invite code

`secrets.token_urlsafe(32)`; stored with `expires_at`, `max_uses`, `used_count`.

PIN join blob

8-digit PIN → PBKDF2 → AES-GCM encrypt of join payload (`group_id`, `group_key`, `metadata`) with AAD `kilid-group-join-v1`.

In-app invite card

`msg_type group_invite` carrying `group_name`, `avatar`, `admin`, optional key material.

10.3 Membership Lifecycle



10.4 Admin Operations

Admins can add/remove members, promote/demote admins, regenerate invite codes, and update group metadata (name, description, avatar). Removing a member does not automatically rotate the `group_key` in the current version; operators should treat departing members as potentially retaining old key material until a future rotation protocol.

10.5 Data Tables Involved

- `groups`
- `group_members`
- `group_invites`
- `group_keys`
- `messages (conversation_type=group)`

11. Runtime, Threading & Reliability

11.1 Threading Model

UI runs on the Qt main thread. DatabaseManager uses an RLock and a connection pool served to worker threads / ThreadPoolExecutor. MQTT network loop runs off the UI thread; callbacks marshal into Qt via signals/slots or QMetaObject.invokeMethod (QueuedConnection). Login/register/restore perform heavy crypto and DB work in daemon threads, then post results to the dialog slots.

11.2 Reconnect Strategy

MAX_RECONNECT_ATTEMPTS = 10 with RECONNECT_DELAYS = [1,2,5,10,15,30,60,120,180,300] seconds. After exhaustion, UI surfaces offline state; user may trigger manual reconnect. HEARTBEAT_INTERVAL = 60 s for presence liveness.

11.3 File & Voice Transfer

Files ≤ 10 MB split into CHUNK_SIZE (128 KB) chunks under public broker payload limits. Voice: PyAudio capture, WAVE storage, optional zlib compression, VOICE_INLINE_MAX_BYTES (96 KB) per MQTT packet with further chunking if needed. Cooldowns prevent spam.

11.4 Maintenance

Background maintenance every CLEANUP_INTERVAL (3600 s): purge expired nonces, sessions, unused invites; delete soft-deleted messages older than 1 year; prune old voice rows (30 days). atexit closes DB pools and flushes logs.

11.5 Logging

LoggerManager singleton → ~/.kilid/logs/kilid.log (50 MB × 10 backups). Named loggers: security, system, audit, performance, network, voice, block. Console + file handlers.

11.6 Error Taxonomy

SecurityError / EncryptionError / CertificateError / AuthenticationError	Crypto & auth failures
DatabaseError	SQLite / pool / migration
NetworkError	MQTT / connectivity
ValidationError	User input / schema rules
VoiceError / BlockError / GroupError	Feature-specific

12. UI Structure & Navigation Map

12.1 Application Shell

After successful login, KilidApp presents a sidebar navigation and a stacked content area. Theme: ModernTheme / qdarkstyle dark glass — backgrounds #0c1023 / #14192d, accent #00d2ff. System tray supports minimize, notifications, and connection status.

12.2 Primary Surfaces

LoginDialog	Frameless modal; modes: login register restore
Conversation list	Contacts + groups, unread, pin/mute, search
Chat surface	Bubbles, composer, attach, voice, reactions, receipts
Groups	Create/join, members, invites, admin tools
Contacts / Profile	Aliases, notes, block list, avatar, Safety Number
Settings	Broker, password change, backup export, transfer, about
Games hub	Mode select, seekers, Backgammon / Chess boards
Notifications	In-app + tray for messages, invites, buzz

12.3 Login / Restore State Machine

login	Default; username + password
register	Confirm password, device name, Terms checkbox
restore chooser	Safety reset Backup file Online transfer
safety reset	Username + 32-digit + new password
file restore	Browse .kilidbak + backup password
online restore	128-char address + 16-digit + broker tab

13. Games Subsystem Architecture

Games are a first-class feature using the same encrypted messaging channel (`msg_type = "game"`). No separate game server is required.

13.1 Components

Games hub page

Mode chips (bot / practice / online), game cards, seek toggle, challenge UI

BackgammonEngine

Board state, dice, legal moves, bear-off, `bot_play`, `to_dict/from_dict`

ChessEngine

Board, turn, `apply_move`, checkmate/draw, `bot_play`, serialization

Board widgets

BackgammonBoardWidget / ChessBoardWidget — paint + input, `move_made` signal

Network actions

invite, accept, decline, roll, move, state, resign, seeking

13.2 Online Match Sequence

1. Optional: seeking=on → broadcast game/lobby seeking to contacts.
2. Challenger sends {game, action:invite, game_id, username}.
3. Peer UI prompts; accept → {action:accept} and color assignment (challenger=0, acceptor=1).
4. Moves / dice / full state serialized in encrypted game packets.
5. Resign sets winner on peer; local announce and disable interaction.

13.3 Local Modes

Bot: timer-driven `bot_play` when `turn ≠ my_color`. Practice: after each turn, `my_color` flips to `engine.turn` so one human plays both sides. Engines reset with fresh `game_id` (`secrets.token_hex(8)`) on start.

14. Threat Model, Limits & Compliance Notes

14.1 Assets

Message plaintext · Identity private keys · Session/group keys · Recovery Safety Number · User password · Local message history · Contact graph

14.2 Adversaries & Guarantees

Network eavesdropper

Cannot read content (E2E). Sees MQTT metadata (topics, timing, size).

Malicious MQTT broker

Same as eavesdropper for content; can drop/delay/reorder packets.

Endpoint malware

Out of scope — can read plaintext and keys once unlocked.

Lost device (locked)

Password + optional OS disk encryption required; Recovery Number offline.

Online guessing

Login attempt logout; PBKDF2 slows offline hash attacks.

14.3 Explicit Non-Goals

KILID does not provide anonymity against global passive adversaries, deniable messaging, or protection if the endpoint is compromised. Public brokers expose social graph metadata. Group key rotation after member removal is not fully automated in v1.0.0.

14.4 Acceptable Use

Terms of Use prohibit terrorism, trafficking, CSAM, fraud, malware distribution, harassment, and use as a dark-market coordination channel. Enforcement may include broker-side blocks and reporting where required by law.

14.5 Document History

Version 1.0.0 — Initial formal architecture release aligned with application VERSION 1.0.0.



KILID Messenger

Software Architecture Document · v1.0.0

<https://kilid.app> · github.com/armanghobadi/kilid